

Investment Portfolio Management System

A Relational Database Project in SQL

Prepared by: Katlego Kgakgamatso

1. Project Description

The goal of this project is to build an investment portfolio management system using SQL, allowing clients to hold one or more portfolios, each containing multiple investments, with returns tracked against those investments over time. The database supports storing client, portfolio, investment, and return records, and answering analytical questions such as total portfolio value, average return by investment type, and top-performing portfolios and clients.

2. Project Objectives

- Design a normalised relational schema capable of representing clients, their portfolios, the investments within each portfolio, and the returns generated by those investments.
- Enforce data integrity between related entities using primary and foreign key constraints.
- Populate the schema with realistic sample data reflecting a Botswana-based client base.
- Write and validate a structured set of SQL queries, ranging from simple retrieval to multi-table joins, aggregation, and correlated subqueries, to answer real analytical questions about portfolio performance.

2.1 Project Features

- **Investment Tracking:** Add and query investments, including type, amount, and date, linked to a specific portfolio.
- **Portfolio Management:** Group multiple investments under named portfolios belonging to a single client.
- **Performance Reporting:** Generate reports covering total portfolio value, return amounts, and return percentages.
- **Investment Analysis:** Compare performance across investment types and portfolios, and identify top performers.
- **Client Management:** Store and query client records, including contact information and their associated portfolios.

3. Tools and Environment

Database: Supabase - a cloud-hosted PostgreSQL platform, used to provision and serve the project's database instance.

Database Client: DBeaver Community Edition - a free, cross-platform desktop SQL client, used to design the schema, load data, and execute and validate all queries against the live database.

Query Language: PostgreSQL (SQL) - covering data definition (CREATE TABLE, constraints), data manipulation (INSERT), and analytical querying (joins, aggregation, subqueries).

4. Database Design

The system is built around four related entities. Each client can hold multiple portfolios; each portfolio can contain multiple investments; and each investment can generate multiple returns over time. This one-to-many chain is enforced throughout the schema using foreign key constraints.

4.1 Entities

Clients: Stores each client's identity and contact details, and serves as the root of the relationship chain.

Portfolios: Stores each portfolio a client holds, linked back to its owning client.

Investments: Stores individual investments held within a portfolio, including type, amount, and date.

Returns: Stores return records generated by an investment, including the return percentage, amount, and date.

4.2 Entity-Relationship Diagram

The diagram below, generated directly from the live schema, shows the four tables and the foreign key relationships connecting them.

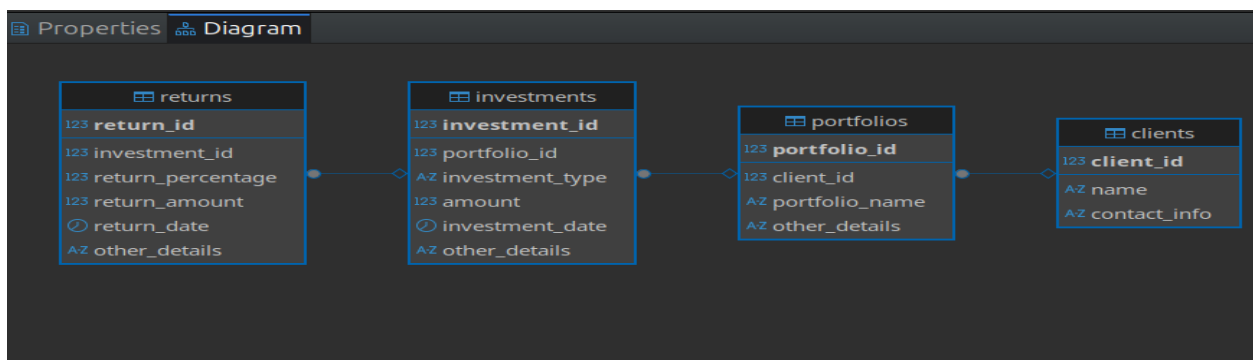
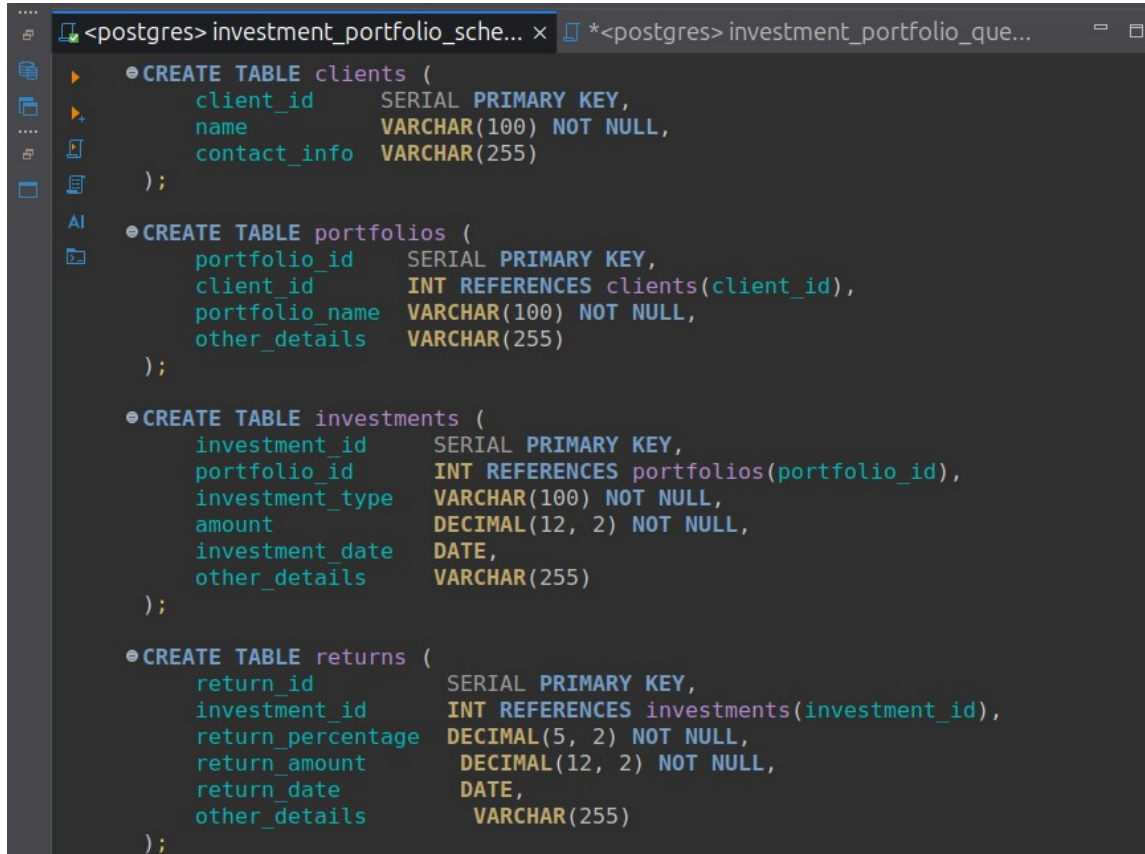


Figure 1: Entity-relationship diagram of the Investment Portfolio Management System schema.

4.3 Schema Definition

The schema below defines all four tables, their columns, and the constraints linking them together.



```
• CREATE TABLE clients (
  client_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  contact_info VARCHAR(255)
);

• CREATE TABLE portfolios (
  portfolio_id SERIAL PRIMARY KEY,
  client_id INT REFERENCES clients(client_id),
  portfolio_name VARCHAR(100) NOT NULL,
  other_details VARCHAR(255)
);

• CREATE TABLE investments (
  investment_id SERIAL PRIMARY KEY,
  portfolio_id INT REFERENCES portfolios(portfolio_id),
  investment_type VARCHAR(100) NOT NULL,
  amount DECIMAL(12, 2) NOT NULL,
  investment_date DATE,
  other_details VARCHAR(255)
);

• CREATE TABLE returns (
  return_id SERIAL PRIMARY KEY,
  investment_id INT REFERENCES investments(investment_id),
  return_percentage DECIMAL(5, 2) NOT NULL,
  return_amount DECIMAL(12, 2) NOT NULL,
  return_date DATE,
  other_details VARCHAR(255)
);
```

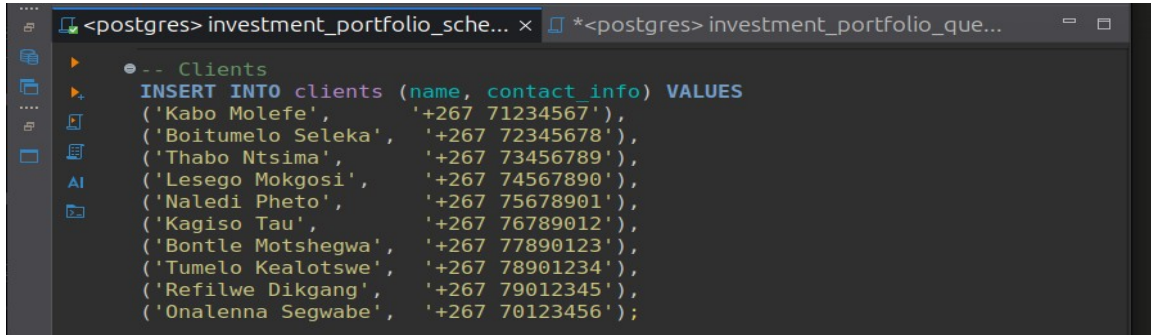
Each table's primary key is generated automatically as new rows are inserted, and every foreign key constraint ensures that a child record can only be created if the parent record it references already exists. This prevents, for example, a portfolio being created for a client that does not exist, or a return being logged against an investment that was never made.

5. Data Population

To simulate real-world usage, the database was populated with ten clients, along with their associated portfolios, investments, and return records.

5.1 Sample Data

The clients were inserted first, as every other table in the schema ultimately depends on a valid client record:



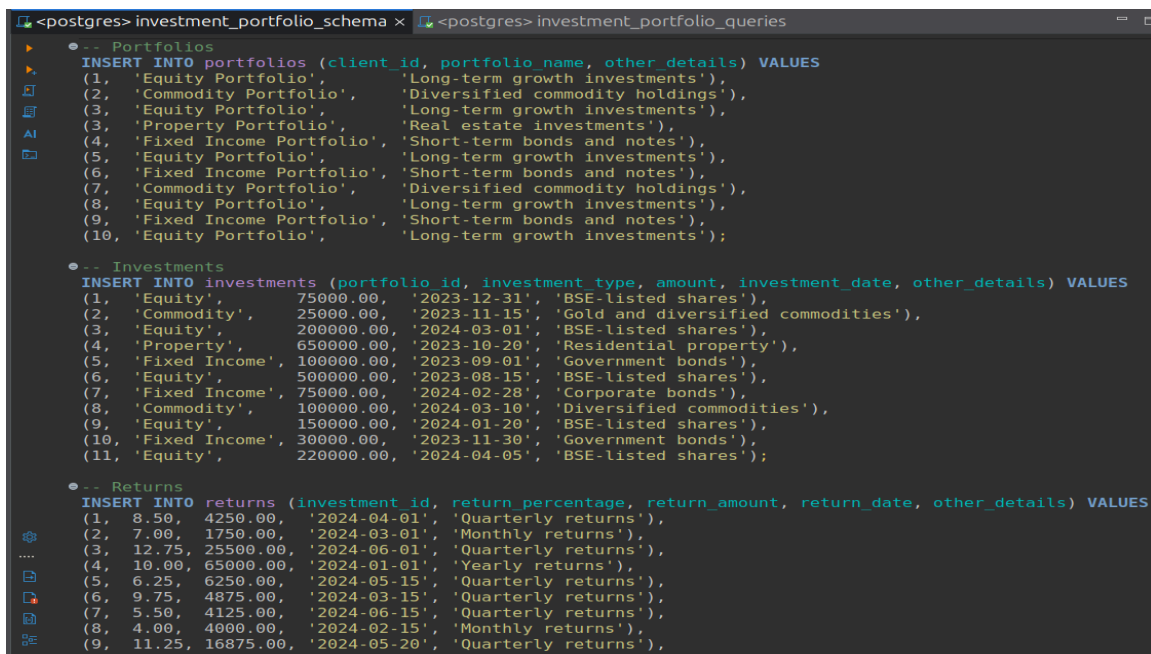
```
-- Clients
INSERT INTO clients (name, contact info) VALUES
('Kabo Molefe', '+267 71234567'),
('Boitumelo Seleka', '+267 72345678'),
('Thabo Ntsima', '+267 73456789'),
('Lesego Mokgosi', '+267 74567890'),
('Naledi Pheto', '+267 75678901'),
('Kagiso Tau', '+267 76789012'),
('Bontle Motshegwa', '+267 77890123'),
('Tumelo Kealotswe', '+267 78901234'),
('Refilwe Dikgang', '+267 79012345'),
('Onalenna Segwabe', '+267 70123456');
```

Figure 2: Inserting the ten sample clients.

Portfolios, investments, and returns were populated in the same way, each new row referencing the identifier of the parent record it belongs to.

5.2 Data Verification

Once loading was complete, row counts were checked across all four tables to confirm the data had been inserted correctly and the relationships were intact:



```
-- Portfolios
INSERT INTO portfolios (client_id, portfolio_name, other details) VALUES
(1, 'Equity Portfolio', 'Long-term growth investments'),
(2, 'Commodity Portfolio', 'Diversified commodity holdings'),
(3, 'Equity Portfolio', 'Long-term growth investments'),
(3, 'Property Portfolio', 'Real estate investments'),
(4, 'Fixed Income Portfolio', 'Short-term bonds and notes'),
(5, 'Equity Portfolio', 'Long-term growth investments'),
(6, 'Fixed Income Portfolio', 'Short-term bonds and notes'),
(7, 'Commodity Portfolio', 'Diversified commodity holdings'),
(8, 'Equity Portfolio', 'Long-term growth investments'),
(9, 'Fixed Income Portfolio', 'Short-term bonds and notes'),
(10, 'Equity Portfolio', 'Long-term growth investments');

-- Investments
INSERT INTO investments (portfolio_id, investment_type, amount, investment_date, other_details) VALUES
(1, 'Equity', 75000.00, '2023-12-31', 'BSE-listed shares'),
(2, 'Commodity', 25000.00, '2023-11-15', 'Gold and diversified commodities'),
(3, 'Equity', 200000.00, '2024-03-01', 'BSE-listed shares'),
(4, 'Property', 650000.00, '2023-10-20', 'Residential property'),
(5, 'Fixed Income', 100000.00, '2023-09-01', 'Government bonds'),
(6, 'Equity', 500000.00, '2023-08-15', 'BSE-listed shares'),
(7, 'Fixed Income', 75000.00, '2024-02-28', 'Corporate bonds'),
(8, 'Commodity', 100000.00, '2024-03-10', 'Diversified commodities'),
(9, 'Equity', 150000.00, '2024-01-20', 'BSE-listed shares'),
(10, 'Fixed Income', 30000.00, '2023-11-30', 'Government bonds'),
(11, 'Equity', 220000.00, '2024-04-05', 'BSE-listed shares');

-- Returns
INSERT INTO returns (investment_id, return_percentage, return_amount, return_date, other_details) VALUES
(1, 8.50, 4250.00, '2024-04-01', 'Quarterly returns'),
(2, 7.00, 1750.00, '2024-03-01', 'Monthly returns'),
(3, 12.75, 25500.00, '2024-06-01', 'Quarterly returns'),
(4, 10.00, 65000.00, '2024-01-01', 'Yearly returns'),
(5, 6.25, 6250.00, '2024-05-15', 'Quarterly returns'),
(6, 9.75, 4875.00, '2024-03-15', 'Quarterly returns'),
(7, 5.50, 4125.00, '2024-06-15', 'Quarterly returns'),
(8, 4.00, 4000.00, '2024-02-15', 'Monthly returns'),
(9, 11.25, 16875.00, '2024-05-20', 'Quarterly returns');
```

Figure 3: Row counts confirming successful data load across all four tables.

6. Query Analysis

Fifteen queries were written against the schema, grouped into three tiers of increasing complexity. Each query is presented alongside the business question it answers.

6.1 Basic Queries

These queries retrieve and summarise data from one or two tables, establishing the foundational joins used throughout the rest of the project.

Query 1: Retrieve all client names and their contact information.

A direct single-table lookup, used as a base reference for client records.

```
SELECT name, contact_info
FROM clients;
```

Query 2: Retrieve all portfolio names along with their corresponding client names.

Joins clients to portfolios to show ownership at a glance.

```
SELECT p.portfolio_name, c.name
FROM portfolios p
JOIN clients c ON p.client_id = c.client_id;
```

Query 3: Retrieve the total investment amount for each portfolio.

Aggregates investment values using SUM and GROUP BY to show portfolio size.

```
SELECT p.portfolio_name, SUM(i.amount) AS total_investment_amount
FROM portfolios p
JOIN investments i ON p.portfolio_id = i.portfolio_id
GROUP BY p.portfolio_name;
```

Query 4: Retrieve all investments made in a specific investment type.

Filters investments by category, here shown for Equity holdings.

```
SELECT i.investment_type, i.amount, p.portfolio_name
FROM investments i
JOIN portfolios p ON i.portfolio_id = p.portfolio_id
WHERE i.investment_type = 'Equity';
```

Query 5: Retrieve the average return percentage for each investment type.

Compares average performance across investment categories using AVG and GROUP BY.

```
SELECT i.investment_type, AVG(r.return_percentage) AS average_return_percentage
FROM investments i
JOIN returns r ON i.investment_id = r.investment_id
```

```
GROUP BY i.investment_type;
```

6.2 Intermediate Queries

These queries introduce multi-table joins across the full relationship chain, ordering, and comparisons against aggregate values.

Query 1: Retrieve the total return amount for each investment type, along with the corresponding portfolio and client information.

Chains all four tables together to trace a return back to the client who ultimately owns it.

```
SELECT i.investment_type, r.return_amount, p.portfolio_name, c.name
FROM returns r
JOIN investments i ON r.investment_id = i.investment_id
JOIN portfolios p ON i.portfolio_id = p.portfolio_id
JOIN clients c ON p.client_id = c.client_id
ORDER BY i.investment_type, r.return_amount DESC;
```

Query 2: Retrieve the top 5 portfolios with the highest total investment amount.

Ranks portfolios by total value and limits the result to the five largest.

```
SELECT p.portfolio_name, SUM(i.amount) AS total_investment_amount
FROM portfolios p
JOIN investments i ON p.portfolio_id = i.portfolio_id
GROUP BY p.portfolio_name
ORDER BY total_investment_amount DESC
LIMIT 5;
```

Query 3: Retrieve the investments made in the past year, along with the corresponding portfolio and client information.

Filters on a rolling date window to surface recent investment activity.

```
SELECT i.investment_type, i.amount, i.investment_date, p.portfolio_name, c.name
FROM investments i
JOIN portfolios p ON i.portfolio_id = p.portfolio_id
JOIN clients c ON p.client_id = c.client_id
WHERE i.investment_date > NOW() - INTERVAL '1 year';
```

Query 4: Retrieve the clients who have investments in multiple portfolios.

Uses COUNT DISTINCT with HAVING to isolate clients diversifying across more than one portfolio.

```
SELECT c.name, COUNT(DISTINCT p.portfolio_id) AS num_portfolios
FROM clients c
JOIN portfolios p ON c.client_id = p.client_id
```

```
JOIN investments i ON p.portfolio_id = i.portfolio_id
GROUP BY c.name
HAVING COUNT(DISTINCT p.portfolio_id) > 1;
```

Query 5: Retrieve the portfolios with a total return amount higher than the average return amount.

Compares each portfolio's total return against a subquery calculating the overall average.

```
SELECT p.portfolio_name, SUM(r.return_amount) AS total_return_amount
FROM portfolios p
JOIN investments i ON p.portfolio_id = i.portfolio_id
JOIN returns r ON i.investment_id = r.investment_id
GROUP BY p.portfolio_name
HAVING SUM(r.return_amount) > (SELECT AVG(return_amount) FROM returns);
```

6.3 Advanced Queries

These queries combine ranking, correlated subqueries, and anti-joins to answer more nuanced portfolio management questions.

Query 1: Retrieve the top 3 clients with the highest total investment amount across all portfolios.

Aggregates investment value at the client level and ranks the results.

```
SELECT c.name, SUM(i.amount) AS total_invested
FROM clients c
JOIN portfolios p ON c.client_id = p.client_id
JOIN investments i ON p.portfolio_id = i.portfolio_id
GROUP BY c.name
ORDER BY total_invested DESC
LIMIT 3;
```

Query 2: Retrieve the portfolios with the highest average return percentage, along with the corresponding investment and client information.

Combines all four tables to rank portfolios by average return performance.

```
SELECT p.portfolio_name, AVG(r.return_percentage) AS avg_return_percentage,
       i.investment_type, c.name
FROM portfolios p
JOIN investments i ON p.portfolio_id = i.portfolio_id
JOIN returns r ON i.investment_id = r.investment_id
JOIN clients c ON p.client_id = c.client_id
GROUP BY p.portfolio_name, i.investment_type, c.name
ORDER BY avg_return_percentage DESC;
```

Query 3: Retrieve the investments that have not yet received any returns.

Uses a LEFT JOIN anti-join pattern to find investments with no matching return record.

```
SELECT i.investment_type, i.amount, i.investment_date, p.portfolio_name, c.name
```

```
FROM investments i
JOIN portfolios p ON i.portfolio_id = p.portfolio_id
JOIN clients c ON p.client_id = c.client_id
LEFT JOIN returns r ON i.investment_id = r.investment_id
WHERE r.return_id IS NULL;
```

Query 4: Retrieve the clients whose every portfolio has at least one investment.

Compares counts of portfolios against counts of invested portfolios to confirm full coverage.

```
SELECT c.name
FROM clients c
JOIN portfolios p ON c.client_id = p.client_id
LEFT JOIN investments i ON p.portfolio_id = i.portfolio_id
GROUP BY c.name
HAVING COUNT(DISTINCT p.portfolio_id) = COUNT(DISTINCT i.portfolio_id);
```

Query 5: Retrieve the single highest-value portfolio for each client.

Uses a correlated subquery to identify, per client, which portfolio holds the greatest total investment.

```
SELECT c.client_id, c.name, p.portfolio_name, SUM(i.amount) AS
total_investment_amount
FROM clients c
JOIN portfolios p ON c.client_id = p.client_id
JOIN investments i ON p.portfolio_id = i.portfolio_id
GROUP BY c.client_id, c.name, p.portfolio_name
HAVING SUM(i.amount) = (
    SELECT MAX(total_investment_amount)
    FROM (
        SELECT c2.client_id, p2.portfolio_id, SUM(i2.amount) AS
total_investment_amount
        FROM clients c2
        JOIN portfolios p2 ON c2.client_id = p2.client_id
        JOIN investments i2 ON p2.portfolio_id = i2.portfolio_id
        GROUP BY c2.client_id, p2.portfolio_id
    ) AS subquery
    WHERE subquery.client_id = c.client_id
);
```

7. Skills Demonstrated

- Relational schema design, including normalisation and enforcing referential integrity through primary and foreign keys.
 - Data population and validation across related tables.
 - Multi-table joins (INNER JOIN and LEFT JOIN) across a four-table relationship chain.
 - Aggregation and grouping using SUM, AVG, and COUNT, combined with GROUP BY and HAVING.
 - Filtering, ranking, and limiting result sets to answer targeted business questions.
 - Correlated and non-correlated subqueries for comparative and per-group analysis.
-

8. Conclusion

This project delivers a complete, working investment portfolio management database: a normalised four-table schema, a realistic sample dataset, and fifteen SQL queries spanning basic retrieval, multi-table analysis, and advanced comparative logic. Together, these demonstrate the ability to design, populate, and query a relational database to answer meaningful business questions in a financial services context, and provide a foundation that could be extended with a reporting layer or dashboard for portfolio performance analysis.